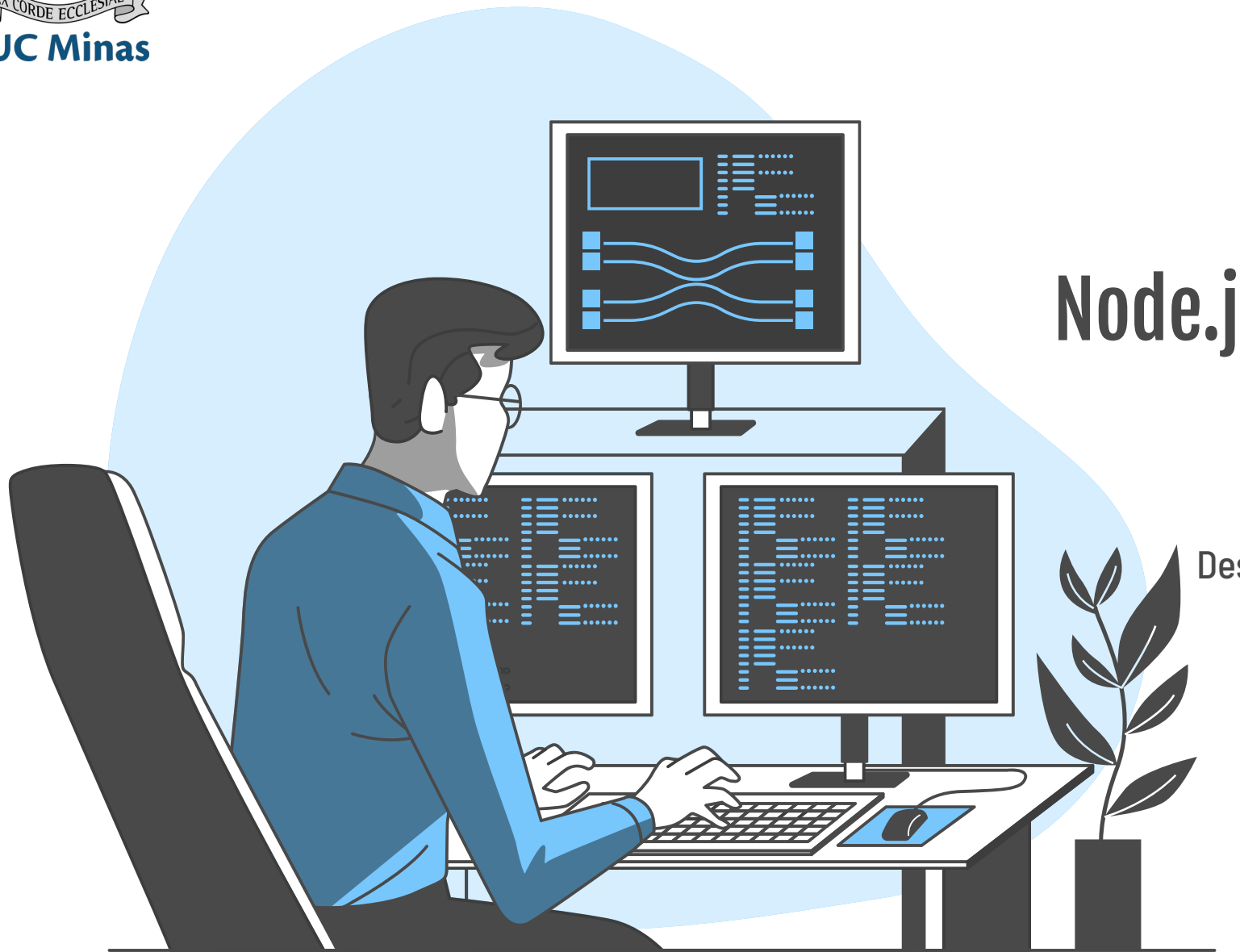
PUC Minas



Aula 01 · Front-end

Node.js, npm e o package.json

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender o papel do **Node.js** no desenvolvimento front-end
2. Saber o que é o **npm**, `npx` e o registro de pacotes
3. Ler um `package.json`: `scripts`, `dependencies`, `devDependencies`
4. Usar `npm install`, `npm run dev`, `npm start` e `npm run build`
5. Entender `node_modules` e `package-lock.json`

Antes de começar

- VS Code instalado (code.visualstudio.com)
- Terminal (PowerShell, Terminal do macOS ou bash)
- Esta é a primeira aula da disciplina: nenhum conhecimento prévio de React é necessário

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Por que Node.js se o React roda no navegador?

02 Instalando

03 O package.json

04 Os comandos do dia a dia

05 node_modules e package-lock.json

06 Prévia: o mesmo papel no Java

07 Aprofundando: o ecossistema npm

01

Por que Node.js se o React roda no navegador?

Por que Node.js se o React roda no navegador?

O código React que você escreve **roda no navegador**. Mas, para escrever esse código com conforto, precisamos de ferramentas que rodam **no seu computador**: um servidor de desenvolvimento que recarrega a página ao salvar, um *bundler* que junta centenas de arquivos em poucos, um *transpilador* que converte JSX em JavaScript. Todas essas ferramentas são programas JavaScript, e o **Node.js** é o programa que executa JavaScript fora do navegador.

01

Por que Node.js se o React roda no navegador?

Aula 01 · Front-end

Por que Node.js se o React roda no navegador?

Peça	O que é
Node.js	*Runtime* que executa JavaScript no computador/servidor
npm	*Node Package Manager*: instala e gerencia bibliotecas. Vem junto com o Node.
npx	Executa um pacote sem instalá-lo de forma permanente
registro npm	O "repositório central" de bibliotecas (npmjs.com)

02

Instalando

Instalando

Baixe a versão **LTS** em **nodejs.org** (o Vite atual exige Node 20.19+ ou 22.12+; use a LTS mais recente). No Linux/macOS, uma alternativa prática é o **nvm** (*Node Version Manager*).



Conferindo as versões do Node.js e do npm.

Instalando

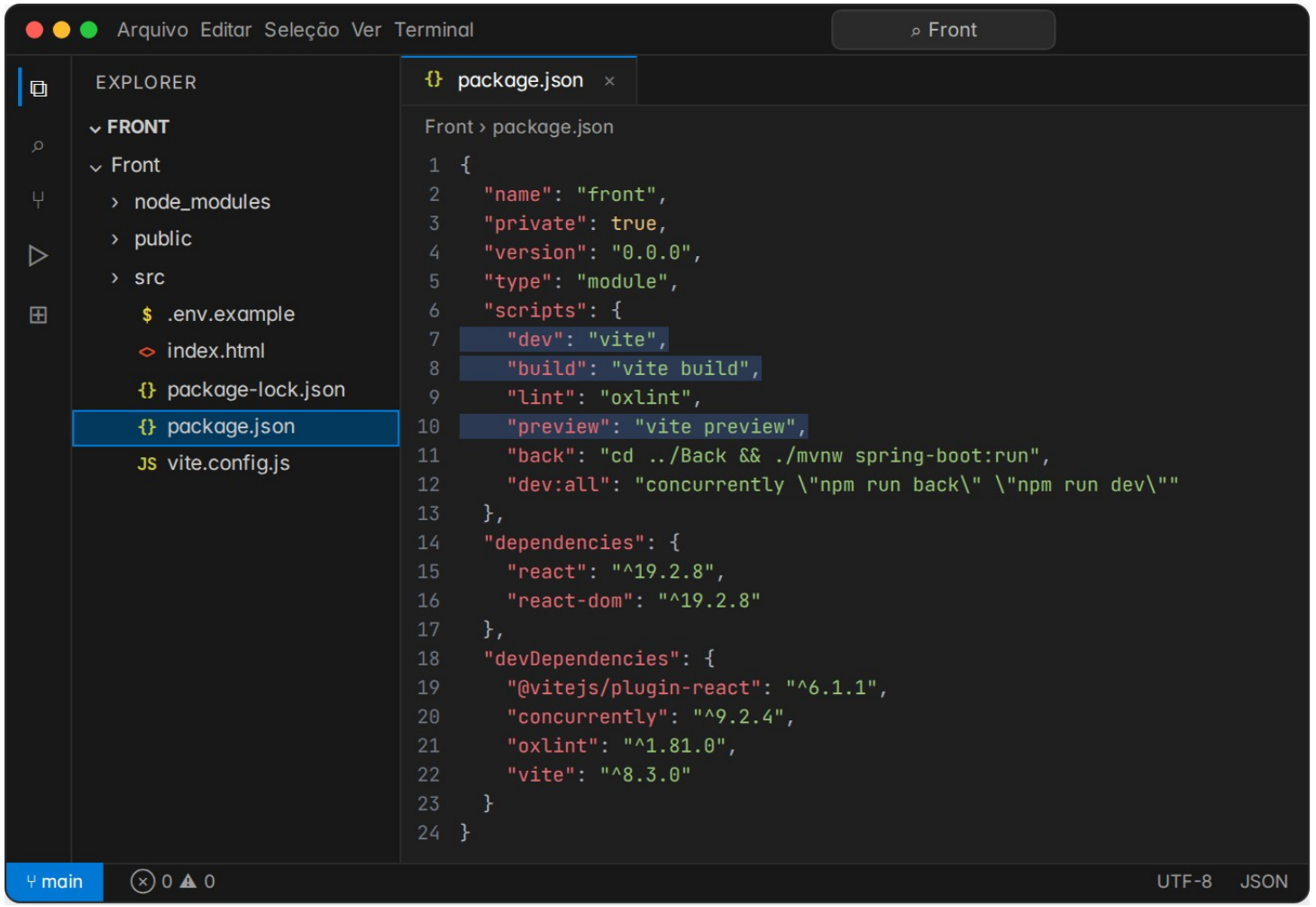
Erro comum

`'npm' não é reconhecido como um comando interno` (Windows): feche e abra o terminal (e o VS Code) depois de instalar. `npm.ps1 cannot be loaded because running scripts is disabled` no PowerShell: rode uma vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` ou use o *Prompt de Comando*.

03

O package.json

O package.json



É a “carteira de identidade” do projeto JavaScript. Fica na raiz do projeto e descreve o nome, os **scripts** (atalhos de comandos) e as **dependências**. Este é o `package.json` do projeto `FullStack/Exemplo_Integracao/Front`:

`package.json` de um projeto React + Vite, com scripts extras para subir o back-end junto (pasta FullStack).

O package.json

Campo	Significado
`"type": "module"`	Usa `import`/`export` (ES Modules) em vez de `require`
`scripts`	Atalhos: `npm run` executa o comando ao lado
`dependencies`	Bibliotecas que vão para o site final (React, React Router, supabase-js)
`devDependencies`	Ferramentas só de desenvolvimento (Vite, linter, testes)
`^19.2.8`	Aceita atualizações **menores** (19.3, 19.4), nunca a 20
`~19.2.8`	Aceita só correções (19.2.9)

04

Os comandos do dia a dia

Os comandos do dia a dia

Comando	O que faz
`npm install` (ou `npm i`)	Lê o `package.json` e baixa **tudo** para `node_modules/`. Rode ao clonar um projeto.
`npm install axios`	Instala um pacote e adiciona em `dependencies`
`npm install -D vitest`	Instala como `devDependencies`
`npm uninstall axios`	Remove o pacote
`npm run dev`	Roda o script `dev` (no Vite: servidor de desenvolvimento)

Os comandos do dia a dia (cont.)

Comando	O que faz
`npm start`	Atalho para o script `start`. **Não precisa do `run`**. No Create React App, sobe o servidor de desenvolvimento.
`npm run build`	Gera a versão de produção (pasta `dist/` no Vite, `build/` no CRA)
`npm run preview`	Serve a pasta `dist/` localmente para conferir o build
`npm ci`	Instala exatamente o que está no `package-lock.json` (usado em Docker e CI)

Os comandos do dia a dia

Observação

Por que `npm start` funciona e `npm dev` não? O npm tem alguns scripts "especiais" que dispensam o `run`: `start`, `test`, `stop` e `restart`. Todos os outros precisam do `run`. Projetos Vite não criam o script `start`; se quiser, adicione `"start": "vite"` ao `package.json`.

Os comandos do dia a dia

Terminal

```
aluno@pc:~/PROJETOS/React/Pokedex$ git clone https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web.git
aluno@pc:~/PROJETOS/React/Pokedex$ cd desenvolvimento-e-integracao-de-aplicacoes-web/PROJETOS/React/Pokedex
aluno@pc:~/PROJETOS/React/Pokedex$ npm install
added 54 packages, and audited 55 packages in 6s
found 0 vulnerabilities
aluno@pc:~/PROJETOS/React/Pokedex$ npm run dev

  VITE v8.3.4  ready in 312 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Fluxo para rodar qualquer projeto React da disciplina: clonar, `npm install`, `npm run dev`.

05

node_modules e package-lock.json

node_modules e package-lock.json

- `node_modules/`: onde ficam os pacotes baixados (pode ter centenas de MB). **Nunca vai para o Git**; o `.gitignore` gerado já exclui. Qualquer pessoa recria com `npm install`.
- `package-lock.json`: registra a **versão exata** de cada pacote instalado (inclusive as dependências das dependências). **Vai para o Git**, para que todos instalem as mesmas versões.

05

node_modules e package-lock.json

Aula 01 · Front-end

node_modules e package-lock.json

Dica

Erro estranho depois de trocar de branch ou atualizar pacotes? O "reiniciar o computador" do Node é apagar e reinstalar:

```
rm -rf node_modules package-lock.json   # Windows: rmdir /s node_modules
npm install
```

06

Prévia: o mesmo papel no Java

Prévia: o mesmo papel no Java

Na pasta Back-end você vai trabalhar com Java e Spring Boot. As ferramentas mudam de nome, mas cumprem os mesmos papéis que você acabou de aprender. Guarde esta tabela: ela vai facilitar a próxima pasta.

Java / Spring (pasta Back-end)	JavaScript / React (esta pasta)
JDK	Node.js
Maven (`mvnw`)	npm
`pom.xml`	`package.json`
Maven Central	registro npm
`~/.m2` / `target/`	`node_modules/` / `dist/`
`./mvnw spring-boot:run`	`npm run dev`
`./mvnw package`	`npm run build`

07

Aprofundando: o ecossistema npm

Versionamento semântico: MAJOR.MINOR.PATCH

MAJOR (19.x.x)

Mudanças que **quebram** compatibilidade. Ex.: React 18 → 19.

MINOR (x.2.x)

Recursos novos, compatíveis com o que existia.

PATCH (x.x.8)

Só correções de bugs.

Prefixos

`^19.2.8` aceita até < 20.0.0; `~19.2.8` até < 19.3.0; `19.2.8` exato.

Aprofundando: o ecossistema npm

Comando	Para que serve
`npm outdated`	Lista pacotes com versões mais novas disponíveis
`npm update`	Atualiza respeitando os prefixos `^` e `~`
`npm list --depth=0`	Mostra os pacotes instalados no projeto
`npm audit`	Procura vulnerabilidades conhecidas
`npm audit fix`	Aplica correções compatíveis
`npm cache clean --force`	Limpa o cache local do npm
`npx`	Executa um pacote sem instalar no projeto

dependencies x devDependencies

dependencies

- Vão para o código que roda no navegador
- `react`, `react-dom`, `react-router-dom`
- `@supabase/supabase-js`, `axios`

vs

devDependencies

- Só ajudam a desenvolver e fazer o build
- `vite`, `@vitejs/plugin-react`
- `oxlint`, `vitest`, `concurrently`

Gerenciadores alternativos

Ferramenta	Instalar dependências	Rodar script	Arquivo de trava
npm	`npm install`	`npm run dev`	`package-lock.json`
yarn	`yarn`	`yarn dev`	`yarn.lock`
pnpm	`pnpm install`	`pnpm dev`	`pnpm-lock.yaml`
bun	`bun install`	`bun run dev`	`bun.lock`

Aprofundando: o ecossistema npm

Atenção

Escolha **um** gerenciador por projeto. Misturar `npm install` e `yarn` gera dois arquivos de trava com versões diferentes, e o Vercel escolhe o gerenciador pelo arquivo de trava que encontrar. Nos projetos da disciplina usamos sempre **npm**.

Aprofundando: o ecossistema npm

Scripts úteis (pre/post rodam automaticamente)

```
{
  "scripts": {
    "dev": "vite",
    "start": "vite",                          // atalho: npm start
    "build": "vite build",
    "preview": "vite preview",
    "lint": "oxlint",
    "test": "vitest",                         // npm test (também dispensa o run)
    "predev": "echo Iniciando o front...", // roda antes do dev
    "postbuild": "echo Build pronto em dist/"
  }
}
```

O arquivo .npmrc

O `.npmrc` na raiz do projeto ajusta o comportamento do npm. O Dockerfile do `PalpiteDTI` copia esse arquivo junto com o `package.json`. Um uso comum é `legacy-peer-deps=true`, que evita erros quando uma biblioteca ainda não declarou suporte à versão mais nova do React.

07

Aprofundando: o ecossistema npm

Aula 01 · Front-end

O arquivo .npmrc

1. Abra o terminal na pasta do projeto (onde está o `package.json`).
2. Rode `npm install` e espere terminar sem erros vermelhos.
3. Rode `npm run dev` e abra o endereço mostrado (`http://localhost:5173`).
4. Para encerrar, volte ao terminal e pressione `Ctrl+C`.
5. Erro estranho? Apague `node_modules` e `package-lock.json` e rode `npm install` de novo.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Por que precisamos do Node.js se o React roda no navegador?

A O React só roda no Node

B As ferramentas de desenvolvimento e build são programas JavaScript

C O navegador exige Node

D Para conectar ao banco

Resposta

Por que precisamos do Node.js se o React roda no navegador?

A. O React só roda no Node

✓ As ferramentas de desenvolvimento e build são programas JavaScript

C. O navegador exige Node

D. Para conectar ao banco

Por quê: Vite, npm e o servidor de desenvolvimento rodam no Node.

Pergunta

Qual comando instala todas as dependências listadas no `package.json`?

A npm run dev

B npm install

C npm start

D npm build

Resposta

Qual comando instala todas as dependências listadas no `package.json`?

A. npm run dev

✓ npm install

C. npm start

D. npm build

Por quê: Ele cria a pasta node_modules.

Quiz · questão 3 de 5

Pergunta

Por que `npm start` funciona sem o `run`?

A Porque start é um script especial do npm

B Porque o Vite cria esse atalho

C Porque é um bug

D Não funciona

Resposta

Por que `npm start` funciona sem o `run`?

✓ Porque start é um script especial do npm

B Porque o Vite cria esse atalho

C Porque é um bug

D Não funciona

Por quê: start, test, stop e restart dispensam o run.

Pergunta

O que não deve ir para o Git?

A package.json

B package-lock.json

C node_modules

D src/

Resposta

O que não deve ir para o Git?

A package.json

B package-lock.json

✓ node_modules

D src/

Por quê: node_modules é recriado com npm install.

Pergunta

Em `"react": "^19.2.8"`, qual versão seria aceita?

A 20.0.0

B 19.4.1

C 18.3.1

D Só 19.2.8

Resposta

Em `"react": "^19.2.8"`, qual versão seria aceita?

A 20.0.0

✓ 19.4.1

C 18.3.1

D Só 19.2.8

Por quê: O ^ aceita MINOR e PATCH, nunca um novo MAJOR.

Fechamento

Resumo da aula

- ✓ O **Node.js** executa as ferramentas de desenvolvimento; o React em si roda no navegador.
- ✓ O **npm** instala pacotes do registro npm; o `npx` executa pacotes sem instalar.
- ✓ O `package.json` guarda **scripts** e **dependências**; `^` aceita versões menores.
- ✓ `npm install` → `npm run dev`. `npm start` dispensa o `run`; `npm run build` gera a produção.
- ✓ `node_modules/` fica fora do Git; `package-lock.json` vai para o Git.

Exercícios

1. Instale o Node LTS e envie o resultado de `node -v` e `npm -v`.

2. Clone o repositório da disciplina e rode o projeto `React/Pokedex`.

3. Adicione o script `"start": "vite"` e confirme que `npm start` funciona.

4. Explique a diferença entre `npm install` e `npm ci`.

Para ir além

- nodejs.org
- docs.npmjs.com/cli/v10/configuring-npm/package-json

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 01?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web